2 · STRATÉGIE CI/CD APPLIQUÉE AUX PIPELINES DE DONNÉES · M2-DE1 · LUNDI 19 OCTOBRE 2026 · 13H40 – 18H10

TD 2 · Le socle : dépôt, conteneurs, première CI

Compétences : C7.2, C8.2 Deck à venir

Objectif

Le socle sur lequel tout le reste se pose : un dépôt bien tenu, des conteneurs, une intégration continue qui construit, teste et publie une image à chaque push. Et une première source lue de bout en bout, pour que la CI teste quelque chose de vrai.

On comprend, on borne

- **Ce qu'une CI garantit** : ça compile, les tests passent, l'image existe et porte un tag qui dit d'où elle vient (`YYYYMMDDHHMMSS-sha`). Rien de plus, mais toujours.
- **La stratégie de branches** : `main` est toujours déployable ; une branche par sujet, une revue (même rapide) avant de merger ; les conventions d'équipe écrites dans `CONTRIBUTING.md` (nommage, format des commits, qui merge).
- **Votre infrastructure en code commence aujourd'hui.** `docker compose up` décrit votre environnement de dev : c'est de l'infrastructure as code. À partir de maintenant, rien de ce qui tourne n'est cliqué ni installé à la main sans être décrit dans le dépôt — l'ordonnancement au TD 3, le déploiement au TD 7, les tableaux de bord au TD 8. Le TD 9 consolidera, il ne commencera pas.
- **On borne** : une CI qui fait lint + tests + build + push d'image. Pas de déploiement automatique aujourd'hui, pas de matrice de versions, pas de cache exotique.

Les objectifs de la séance

Dans cet ordre, à votre rythme : le temps est à vous, le livrable est pour la fin.

1. Rappel : ce qu'on attend d'une CI, démonstration d'un pipeline qui tourne, revue rapide des notes de cadrage.
2. **L'arborescence** (`src/` , `tests/` , `docs/` , `infra/`), le `Dockerfile` (multi-étapes, image finale petite, utilisateur non root), `docker-compose.yml` (l'app + Postgres + ce dont vous avez besoin), `bin/dev` et `bin/test` . Objectif : un·e nouveau·elle clone et lance en deux commandes.
3. **La première source** : lire ce qu'une source publie (une seule, choisie dans `sources.csv`), le parser, écrire les items en JSON dans un fichier. Un test unitaire sur le parseur avec un exemple enregistré dans `tests/fixtures/` .
4. **Le pipeline CI** (GitHub Actions ou GitLab CI) : lint, tests, build de l'image, push dans le registre (ghcr.io ou le registre GitLab) avec le tag horodaté + sha. Vérifiez qu'un test cassé rend la CI rouge.
5. `README.md` (lancer, tester, contribuer), `CONTRIBUTING.md` , revue croisée entre deux équipes : chacune clone l'autre et lance en deux commandes. Ce qui ne marche pas chez l'autre est votre premier bug.

Ce qu'on construit

- `Dockerfile` , `docker-compose.yml` , `bin/dev` , `bin/test` .
- Un module qui lit une source et produit du JSON, avec son test et sa fixture.
- `.github/workflows/ci.yml` (ou `.gitlab-ci.yml`) : lint → tests → build → push.
- `README.md` , `CONTRIBUTING.md` , et un premier `docs/ci-cd.md` : la stratégie CI/CD retenue, en dix lignes.

Où on doit en être à la fin

Un push sur `main` déclenche une CI verte qui publie une image taguée ; une CI rouge bloque le merge ; une personne extérieure lance le projet en deux commandes depuis le README.

Pour la prochaine séance

- Toute l'équipe a fait au moins un commit qui est passé par la CI.
- Regardez de près la façon dont vos sources publient (le format, ses variantes) et les en-têtes HTTP qu'elles renvoient (`curl -I`) : vous en aurez besoin au TD 3.

Compétences visées

C7.2 — des pipelines qui livrent vite et de façon fiable, des tests intégrés, moins d'erreurs humaines.
C8.2 — l'infrastructure de dev décrite en code (Compose).

LIVRABLE : Un push déclenche une CI verte (lint, tests, build) qui publie une image ; le README lance le projet en deux commandes

← TD 1 · Cadrage et bornage

TD 3 · Collecter les flux →